

JAVA PROGRAMMING

UNIT – II

IT405PC Java Programming

Exception Handling | Files and I/O Streams



TABLE OF CONTENTS

- Part A: Exception Handling
 - 1. Exception and Error
 - 2. Exception Types
 - 3. Exception Handler
 - 4. Exception Handling Clauses
 - 5. Built-in and Custom Exceptions
- Part B: Files and I/O Streams
 - 1. The File Class
 - 2. Streams Overview
 - 3. Byte Streams
 - 4. Filtered Byte Streams
 - 5. Random Access File Class

EXCEPTION - DEFINITION

■ Definition:

- An exception is an abnormal event or condition that disrupts the normal flow of a program's execution during runtime.
- An error is a serious problem that a reasonable application should not try to catch, usually related to the environment in which the application is running.

■ Key Difference:

- Exceptions → Can be handled (recoverable)
- Errors → Should not be handled (non-recoverable)

EXCEPTION VS ERROR

- **Exception:**

- Represents conditions that a program can anticipate and recover from
- Examples: File not found, invalid user input, network timeout
- Occurs during program execution (runtime)
- Can be caught and handled using exception handling mechanisms
- Application can continue execution after handling

- **Error:**

- Represents serious problems related to the Java runtime environment
- Examples: OutOfMemoryError, StackOverflowError
- Usually beyond the control of the programmer
- Application typically cannot recover from errors
- Indicates critical problems with the JVM or system

EXCEPTION HIERARCHY

```
java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Error
│   │   ├── VirtualMachineError
│   │   ├── OutOfMemoryError
│   │   └── StackOverflowError
│   └── java.lang.Exception
│       ├── IOException
│       ├── SQLException
│       ├── ClassNotFoundException
│       └── RuntimeException
│           ├── NullPointerException
│           ├── ArithmeticException
│           ├── ArrayIndexOutOfBoundsException
│           └── NumberFormatException
```

EXCEPTION TYPES - DEFINITION

- Two Main Types:
 1. Checked Exceptions (Compile-time Exceptions)
 - Must be declared or handled explicitly
 - Checked by compiler at compile-time
 2. Unchecked Exceptions (Runtime Exceptions)
 - Not checked at compile-time
 - Occur during program execution

EXCEPTION TYPES

- **Checked Exceptions:**
 - Extend `java.lang.Exception` (but not `RuntimeException`)
 - Compiler forces you to handle them
 - Used for recoverable conditions
 - Examples: `IOException`, `SQLException`, `FileNotFoundException`
 - **MUST** use try-catch or declare with throws
- **Unchecked Exceptions:**
 - Extend `java.lang.RuntimeException`
 - Compiler does not enforce handling
 - Usually indicate programming errors
 - Examples: `NullPointerException`, `ArithmeticException`
 - Optional to handle (but recommended for critical code)

EXCEPTION HANDLER - DEFINITION

- Definition:
 - Exception Handler is a block of code designed to process (handle) exceptions when they occur, preventing program termination and allowing graceful error recovery.
- Components:
 - try block - Contains code that might throw an exception
 - catch block - Handles the exception
 - finally block - Executes regardless of exception

EXCEPTION HANDLING MECHANISM

■ How It Works:

1. 1. Normal Execution: Code runs in try block
2. 2. Exception Occurs: Abnormal condition detected
3. 3. Exception Object Created: JVM creates exception object
4. 4. Stack Unwinding: JVM searches for appropriate handler
5. 5. Handler Found: Matching catch block executes
6. 6. Finally Executes: Cleanup code runs
7. 7. Program Continues: Execution resumes after try-catch-finally

■ Benefits:

- Separates error-handling code from regular code
- Propagates errors up the call stack
- Groups error types
- Provides meaningful error information

TRY BLOCK - SYNTAX

- Definition:

- The try block encloses code that might throw an exception.

```
try{  
    // Code that may throw an exception  
    // Protected code  
}
```

- Rules:

- Must be followed by either catch or finally (or both)
- Can contain multiple statements
- Can have nested try blocks
- Cannot exist alone

CATCH BLOCK - SYNTAX

- Definition:
 - The catch block handles specific exceptions thrown by the try block.

Syntax:

```
-----  
catch (ExceptionType referenceVariable) {  
    // Exception handling code  
}
```

Multiple Catch Blocks:

```
try {  
    // code  
} catch (IOException e) {  
    // Handle IOException  
} catch (SQLException e) {  
    // Handle SQLException  
} catch (Exception e) {  
    // Handle any other exception  
}
```

CATCH BLOCK

- Key Points:

1. Order Matters:

- Specific exceptions first, general exceptions last
- Subclass before superclass

2. Multi-catch (Java 7+):

```
catch (IOException | SQLException e) {  
    // Handle both exceptions  
}
```

3. Exception Object Methods:

- e.getMessage() - Returns error message
- e.printStackTrace() - Prints stack trace
- e.toString() - Returns exception description

FINALLY BLOCK - SYNTAX

- Definition:

- The finally block contains code that executes regardless of whether an exception occurs or not, typically used for cleanup operations.

- Common Uses:

- Closing file streams
- Releasing database connections
- Freeing resources
- Cleanup operations

Syntax:

```
-----  
try {  
    // Protected code  
} catch (ExceptionType e) {  
    // Exception handling  
} finally {  
    // Cleanup code - ALWAYS executes  
}
```

FINALLY BLOCK

- Execution Scenarios:
 1. No exception: try → finally
 2. Exception caught: try → catch → finally
 3. Exception not caught: try → finally → propagate exception
 4. Return in try/catch: finally executes BEFORE return
- Important Notes:
 - Executes even if try/catch has a return statement
 - Does NOT execute if JVM exits (System.exit())
 - Does NOT execute if thread dies
 - Can exist with try without catch (try-finally)

THROWS KEYWORD - SYNTAX

- Definition:
 - The throws keyword declares that a method may throw one or more exceptions, delegating exception handling to the calling method.

Syntax:

```
returnType methodName(parameters) throws ExceptionType1, ExceptionType2 {  
    // Method body  
}
```

Example:

```
public void readFile(String filename)  
    throws IOException, FileNotFoundException {  
    // Code that may throw these exceptions  
}
```

THROWS KEYWORD

■ Key Points:

1. Declaration: Informs caller about potential exceptions
2. Propagation: Passes exception responsibility to caller
3. Multiple Exceptions: Can declare multiple exceptions
4. Checked Exceptions: Mandatory for checked exceptions
5. Unchecked Exceptions: Optional for unchecked exceptions

Usage Pattern:

Method A throws Exception

↓

Method B calls A (must handle or throw)

↓

Method C calls B (must handle or throw)

THROW STATEMENT - SYNTAX

- Definition:
 - The throw statement explicitly throws an exception object from a method or block of code.

Syntax:

```
throw new ExceptionType("Error message");
```

Examples:

```
throw new ArithmeticException("Division by zero");
```

```
throw new IllegalArgumentException("Age cannot be negative");
```

```
throw new IOException("File not found");
```

THROW VS THROWS

throw	throws
Used to explicitly throw an exception	Used to declare exceptions
Used inside method body	Used in method signature
Followed by an instance	Followed by class name(s)
Can throw only one at a time	Can declare multiple exceptions
<code>throw new Exception();</code>	<code>throws IOException, SQLException</code>

BUILT-IN EXCEPTIONS - COMMON TYPES

- Checked Exceptions:
 1. IOException - I/O operation failed
 2. FileNotFoundException - File doesn't exist
 3. SQLException - Database access error
 4. ClassNotFoundException - Class not found
 5. InterruptedException - Thread interrupted
- Unchecked Exceptions (RuntimeException):
 1. NullPointerException - Null object reference
 2. ArithmeticException - Arithmetic error (e.g., divide by zero)
 3. ArrayIndexOutOfBoundsException - Invalid array index
 4. NumberFormatException - Invalid number format
 5. IllegalArgumentException - Invalid method argument

CUSTOM EXCEPTIONS - DEFINITION

- Definition:
 - Custom Exceptions are user-defined exception classes created to handle specific error conditions relevant to your application domain.
- Why Create Custom Exceptions?
 - Represent application-specific errors
 - Provide meaningful error messages
 - Better error categorization
 - Improve code readability
 - Easier debugging and maintenance

CUSTOM EXCEPTIONS - SYNTAX

Creating Custom Checked Exception:

```
public class CustomCheckedException extends Exception {  
    public CustomCheckedException() {  
        super();  
    }  
  
    public CustomCheckedException(String message) {  
        super(message);  
    }  
  
    public CustomCheckedException(String message, Throwable cause) {  
        super(message, cause);  
    }  
}
```

Creating Custom Unchecked Exception:

```
public class CustomUncheckedException extends RuntimeException {  
    public CustomUncheckedException(String message) {  
        super(message);  
    }  
}
```

COMPLETE EXCEPTION HANDLING EXAMPLE

```
// 1. Defining a Custom Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class ExceptionHandlingDemo {

    // 2. Using 'throws' to declare that this method might throw an exception
    public static void checkEligibility(int age) throws InvalidAgeException {
        if (age < 18) {
            // 3. Using 'throw' to explicitly trigger an exception
            throw new InvalidAgeException("Age must be 18+ to vote.");
        } else {
            System.out.println("Welcome to the voting system.");
        }
    }
}
```

COMPLETE EXCEPTION HANDLING EXAMPLE

```
public static void main(String[] args) {  
    try {  
        // Scenario A: Arithmetic Exception (Unchecked)  
        int result = 10 / 0;  
        // Scenario B: Custom Exception (Checked)  
        checkEligibility(15);  
    } catch (ArithmeticException e) {  
        // 4. Catching specific built-in exceptions first  
        System.out.println("Error: Cannot divide by zero! " + e.getMessage());  
    } catch (InvalidAgeException e) {  
        // 5. Catching custom exceptions  
        System.out.println("Eligibility Error: " + e.getMessage());  
    } catch (Exception e) {  
        // 6. Generic catch block for any other unforeseen exceptions  
        System.out.println("An unexpected error occurred: " + e.getMessage());  
    } finally {  
        // 7. Finally block always executes  
        System.out.println("Execution complete. Cleaning up resources...");  
    }  
    System.out.println("Program continues normally after handling  
exceptions.");  
}
```

THE FILE CLASS - DEFINITION

- Definition:
 - The File class (from java.io package) represents file and directory pathnames in an abstract manner, providing methods to create, delete, and retrieve information about files and directories.
- Important Note:
 - File class does NOT handle file content (reading/writing)
 - It only represents the pathname and metadata
 - Used for file and directory manipulation operations
- Package:
 - `import java.io.File;`

THE FILE CLASS - CONSTRUCTORS SYNTAX

```
// Constructor 1: Using pathname string
File file1 = new File(String pathname);

// Constructor 2: Using parent path and child name
File file2 = new File(String parent, String child);

// Constructor 3: Using parent File object and child name
File file3 = new File(File parent, String child);

// Constructor 4: Using URI
File file4 = new File(URI uri);
```

Examples:

```
File file1 = new File("test.txt");
File file2 = new File("C:\\Users\\Documents", "test.txt");
File parent = new File("C:\\Users");
File file3 = new File(parent, "Documents\\test.txt");
```

THE FILE CLASS - IMPORTANT METHODS

File Information Methods:

1. `boolean exists()` - Check if file/directory exists
2. `boolean isFile()` - Check if it's a file
3. `boolean isDirectory()` - Check if it's a directory
4. `boolean canRead()` - Check read permission
5. `boolean canWrite()` - Check write permission
6. `long length()` - Get file size in bytes
7. `long lastModified()` - Get last modification time
8. `String getName()` - Get file/directory name
9. `String getPath()` - Get path as string
10. `String getAbsolutePath()` - Get absolute path

File Operation Methods:

1. `boolean createNewFile()` - Create new file
2. `boolean mkdir()` - Create directory
3. `boolean delete()` - Delete file/directory
4. `String[] list()` - List files/directories

STREAMS - DEFINITION

- Definition:
 - A Stream is a sequence of data elements that supports sequential operations, providing an abstraction for reading from or writing to a source (file, network, memory) in a uniform manner.
- Two Categories:
 1. Byte Streams - Handle I/O of raw binary data (8-bit bytes)
 2. Character Streams - Handle I/O of character data (16-bit Unicode)
- Direction:
 - Input Stream - Reading data from source
 - Output Stream - Writing data to destination

BYTE STREAMS - DEFINITION

- Definition:
 - Byte Streams perform input and output of 8-bit bytes, used for handling raw binary data such as images, audio files, serialized objects, or any non-text data.
- Base Classes:
 1. InputStream - Abstract class for reading bytes
 2. OutputStream - Abstract class for writing bytes
- Common Byte Stream Classes:
 - FileInputStream / FileOutputStream
 - ByteArrayInputStream / ByteArrayOutputStream
 - BufferedInputStream / BufferedOutputStream
 - DataInputStream / DataOutputStream

FILEINPUTSTREAM AND FILEOUTPUTSTREAM

FileInputStream Constructors:

```
FileInputStream(String name)           // Using filename
FileInputStream(File file)             // Using File object
FileInputStream(FileDescriptor fdObj)  // Using file descriptor
```

FileOutputStream Constructors:

```
FileOutputStream(String name)          // Create/overwrite file
FileOutputStream(String name, boolean append) // Append mode
FileOutputStream(File file)            // Using File object
FileOutputStream(File file, boolean append) // Append mode
```

Example Usage:

```
FileInputStream fis = new FileInputStream("input.txt");
FileOutputStream fos = new FileOutputStream("output.txt");
FileOutputStream fos2 = new FileOutputStream("output.txt", true); // Append
```

FILTERED BYTE STREAMS

- **Definition:**
 - Filtered Byte Streams are wrapper streams that add functionality to existing byte streams by filtering or transforming data as it's read or written.
- **Base Classes:**
 - `FilterInputStream` - Wraps `InputStream`
- **Common Filtered Streams:**
 1. `BufferedInputStream` / `BufferedOutputStream` - Add buffering
 2. `DataInputStream` / `DataOutputStream` - Read/write primitive types
 3. `PushbackInputStream` - Push back bytes to stream
 4. `PrintStream` - Formatted output `FilterOutputStream` - Wraps `OutputStream`

BUFFEREDINPUTSTREAM AND BUFFEREDOUTPUTSTREAM

- Definition:

- Buffered streams add a memory buffer to reduce the number of actual I/O operations, improving performance significantly.

Syntax:

```
-----  
// BufferedInputStream  
BufferedInputStream(InputStream in)  
BufferedInputStream(InputStream in, int size) // Custom buffer size  
  
// BufferedOutputStream  
BufferedOutputStream(OutputStream out)  
BufferedOutputStream(OutputStream out, int size)
```

Example:

```
-----  
FileInputStream fis = new FileInputStream("data.txt");  
BufferedInputStream bis = new BufferedInputStream(fis);  
  
FileOutputStream fos = new FileOutputStream("output.txt");  
BufferedOutputStream bos = new BufferedOutputStream(fos);
```

Important Methods:

```
-----  
void flush() // Force buffer contents to be written
```

DATAINPUTSTREAM AND DATAOUTPUTSTREAM

■ Definition

- Data streams allow reading and writing of Java primitive data types in a machine-independent way.

Syntax:

```
-----  
DataInputStream(InputStream in)  
DataOutputStream(OutputStream out)
```

DataInputStream Methods	DataOutputStream Methods
boolean readBoolean()	void writeBoolean(boolean v)
byte readByte()	void writeByte(int v)
short readShort()	void writeShort(int v)
int readInt()	void writeInt(int v)
long readLong()	void writeLong(long v)
float readFloat()	void writeFloat(float v)
double readDouble()	void writeDouble(double v)
char readChar()	void writeChar(int v)
String readUTF()	void writeUTF(String str)

RANDOM ACCESS FILE - CLASS

- **Definition:**

- RandomAccessFile is a class that allows reading from and writing to a file at any position (non-sequential access), supporting both input and output operations on the same file object.

- **Key Characteristics:**

- Random Access: Can jump to any position in file
- Bidirectional: Both read and write in same object
- File Pointer: Maintains current position in file
- Not a Stream: Does not extend InputStream or OutputStream
- Implements: DataInput and DataOutput interfaces

- **Package:**

- `import java.io.RandomAccessFile;`

RANDOM ACCESS FILE - SYNTAX

Constructors:

```
RandomAccessFile(String name, String mode)
```

```
RandomAccessFile(File file, String mode)
```

Access Modes:

"r" - Read only

"rw" - Read and write

"rws" - Read, write, synchronous (content + metadata)

"rwd" - Read, write, synchronous (content only)

Examples:

```
// Read-only mode
```

```
RandomAccessFile raf1 = new RandomAccessFile("data.dat", "r");
```

```
// Read-write mode
```

```
RandomAccessFile raf2 = new RandomAccessFile("data.dat", "rw");
```

```
// Using File object
```

```
File file = new File("data.dat");
```

```
RandomAccessFile raf3 = new RandomAccessFile(file, "rw");
```

RANDOM ACCESS FILE

- File Pointer Methods:
 - `long getFilePointer()` - Get current position
 - `void seek(long pos)` - Move to position
 - `long length()` - Get file length
 - `void setLength(long newLength)` - Set file length
- Reading Methods (from `DataInput`):
 - `int read()` - Read single byte
 - `boolean readBoolean()`, `byte readByte()`
 - `int readInt()`, `long readLong()`
 - `double readDouble()`, `String readUTF()`
- Writing Methods (from `DataOutput`):
 - `void write(int b)` - Write single byte
 - `void writeBoolean(boolean v)`, `void writeInt(int v)`
 - `void writeLong(long v)`, `void writeDouble(double v)`
 - `void writeUTF(String str)`

SUMMARY - EXCEPTION HANDLING

■ Key Concepts:

1. Exceptions vs Errors: Recoverable vs Non-Recoverable
2. Exception Types: Checked vs Unchecked
3. Handling Mechanism: try-catch-finally
4. Propagation: throw and throws keywords
5. Custom Exceptions: Domain-specific error handling

■ Best Practices:

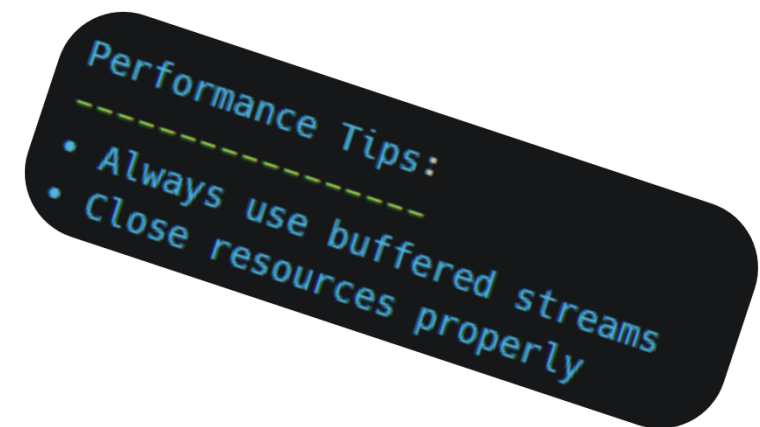
- Catch specific exceptions
- Always clean up resources (use finally or try-with-resources)
- Don't swallow exceptions
- Create meaningful custom exceptions
- Document exceptions with @throws

■ Benefits:

1. Separates error handling from normal code
2. Centralized error management

SUMMARY - FILES AND I/O STREAMS

- **File Class:**
 - Represents file/directory pathnames
 - Metadata and file operations
 - No content reading/writing
- **Stream Types:**
 - Byte Streams: Binary data (InputStream/OutputStream)
 - Character Streams: Text data (Reader/Writer)
 - Filtered Streams: Add functionality (Buffered, Data)
- **RandomAccessFile:**
 - Non-sequential file access
 - Read and write in same object
 - seek() for positioning
 - Perfect for database-like files



BEST PRACTICES

-  **Good Practices:**
 - Use try-with-resources for automatic resource management
 - Handle specific exceptions before general ones
 - Always use buffered streams for better performance
 - Close all streams and files properly
 - Use meaningful exception messages
 - Log exceptions appropriately
-  **Anti-Patterns to Avoid:**
 - Empty catch blocks (swallowing exceptions)
 - Catching Throwable or Error
 - Not closing streams (resource leaks)
 - Using exceptions for flow control
 - Unbuffered operations on large files

QUICK REFERENCE - STREAM CLASSES

Class	Purpose	Common Use
FileInputStream FileOutputStream BufferedInputStream BufferedOutputStream DataInputStream DataOutputStream RandomAccessFile	Read bytes from file Write bytes to file Buffered byte reading Buffered byte writing Read primitive types Write primitive types Random access I/O	Binary reading Binary writing Performance Performance Structured data Structured data Database files